

Rapport de TP 6 : Algorithme de Dijkstra Parallèle

Analyse comparative d'implémentations séquentielles et parallèles avec OpenMP

Gabriel Mazet - N° d'étudiant : 22202954

Rappel du Sujet et Objectifs

L'objectif de ce TP est de concevoir, implémenter et paralléliser l'algorithme de Dijkstra à l'aide de la bibliothèque **OpenMP**. L'algorithme de Dijkstra permet de résoudre le problème du plus court chemin à source unique dans un graphe orienté ou non orienté dont les arcs sont pondérés par des valeurs positives ou nulles (distances).

Le travail s'intéresse ici à la recherche du plus court chemin entre le premier nœud du graphe (nœud n°1, indexé 0 en C) et tous les autres nœuds. Deux topologies de réseaux routiers issus de la *9ème Challenge DIMACS* sont fournies pour évaluer les implémentations :

- rome99.gr** : un réseau de taille modérée représentant le réseau routier de Rome en 1999, comportant **3 353** nœuds et **8 870** arcs.
- new-york.gr** : un réseau de grande taille représentant la ville de New York, comportant **264 346** nœuds et **733 846** arcs. Ce fichier est particulièrement adapté pour mesurer l'accélération et le passage à l'échelle (scaling) des implémentations OpenMP.

À partir du code séquentiel fourni (**dijkstra_seq2.c**), mon travail consiste à :

- Comprendre la structure globale du code et analyser ses forces et faiblesses.
- Proposer une parallélisation OpenMP directe de l'algorithme fourni.
- Identifier les limites structurelles de la complexité algorithmique proposée et concevoir une version optimisée, puis paralléliser cette dernière.
- Conduire une analyse de performance rigoureuse sur différentes configurations de cœurs, en analysant la pertinence des indicateurs statistiques (moyenne, minimum, médiane).

Analyse du Code Séquentiel et Optimisation Algorithmique

Analyse de la version naïve fournie

La version séquentielle initiale utilise une structure classique : les nœuds explorés sont marqués à l'aide d'un tableau de booléens **P** (pour *Permanents*) de taille V (nombre de nœuds), et les distances provisoires sont stockées dans un tableau **d** de taille V.

L'algorithme se déroule en V - 1 étapes. À chaque étape, il effectue deux opérations majeures :

- Recherche du nœud le plus proche** : On parcourt l'ensemble des V nœuds pour identifier celui qui possède la plus petite distance provisoire **d[i]** non encore permanente (**!P[i]**). Cette étape s'effectue en temps linéaire O(V) par simple balayage.
- Mise à jour des distances** : Une fois le nœud le plus proche (nommé **nearest_node**) identifié, on met à jour la distance provisoire des nœuds restants. Dans la boucle fournie, cette mise à jour s'effectue ainsi :

```
// Extrait du code séquentiel fourni
for (int i = 0; i < num_nodes; i++) {
    if ( !P[i] ) {
        int dist = get_distance(nearest_node, i);
        if ( dist < INFINITE ) {
            if ( d[nearest_node] + dist < d[i] ) {
                d[i] = d[nearest_node] + dist;
            }
        }
    }
}
```

⚠ Analyse critique de la complexité naïve

Cette boucle de mise à jour s'exécute V fois. À chaque itération, elle appelle la fonction `get_distance(nearest_node, i)`. Comme le graphe est modélisé par une liste d'adjacence (tableau de listes chaînées `nodes`), `get_distance` doit parcourir la liste chaînée des voisins de `nearest_node` pour vérifier s'il existe un arc vers le nœud `i`.

Le coût de `get_distance` est proportionnel au degré sortant du nœud, noté $\deg(u)$. Ainsi, le coût total de cette étape de mise à jour pour une seule itération externe est de $O(V \cdot \deg(\text{nearest_node}))$. Sur l'ensemble de l'algorithme, la complexité de cette version naïve s'élève donc à :

$$T_{\text{naïve}} = O(V^2 + V \cdot E)$$

Où $E = \sum \deg(u)$ est le nombre total d'arcs. Avec un réseau routier comme New York ($V \approx 264k$, $E \approx 733k$), le terme $V \cdot E$ représente environ $1,9 \times 10^{11}$ opérations ! C'est ce qui explique la lenteur extrême du code initial.

Optimisation Séquentielle : Exploitation des listes d'adjacence

Dans un graphe creux (comme un réseau routier, où le degré moyen d'un nœud est très faible, typiquement entre 2 et 4), il est inutile de parcourir tous les nœuds i de 0 à $V - 1$ pour mettre à jour les distances. Seuls les voisins directs de `nearest_node` peuvent voir leur distance modifiée !

J'ai donc réécrit l'étape de mise à jour des distances en parcourant uniquement la liste chaînée des arcs sortants du nœud courant. L'initialisation a également été optimisée pour ne visiter que les voisins directs de la source (le nœud 0) au lieu de balayer le graphe complet :

```
// Code optimisé pour la mise à jour des voisins
struct direct_edge_struct *edge = nodes[nearest_node];
while (edge != NULL) {
    int dest = edge->destination_node;
    if (!P[dest]) {
        if (d[nearest_node] + edge->weight < d[dest]) {
            d[dest] = d[nearest_node] + edge->weight;
        }
    }
    edge = edge->next;
}
```

La complexité de cette phase de mise à jour sur l'ensemble de l'exécution tombe à $O(E)$ puisque chaque arc du graphe est examiné exactement une fois (lors du passage du nœud source correspondant dans le sous-graphe permanent). La complexité théorique totale passe alors à :

$$T_{\text{optimisé}} = O(V^2 + E)$$

Pour New York, le terme dominant est désormais $V^2 \approx 7 \times 10^{10}$ opérations de recherche de minimum, ce qui représente une réduction massive du travail par rapport à la version naïve.

Détection et correction d'une divergence physique (Arcs parallèles)

Lors de la validation de mes implémentations sur le graphe de Rome, j'ai constaté une divergence dans le calcul du chemin le plus court entre la version naïve d'origine et la version optimisée par voisinage. En analysant le fichier de graphe, j'ai découvert qu'il s'agissait d'un réseau routier bidirectionnel où certains nœuds sont reliés par des **arcs parallèles multiples** (plusieurs lignes `a u v w` pour les mêmes nœuds, représentant par exemple des voies différentes ou des arcs asymétriques).

Dans la version séquentielle d'origine, la fonction `get_distance(u, v)` renvoie la pondération du **premier** arc correspondant trouvé dans la liste d'adjacence. S'il existe un second arc parallèle plus court, il était ignoré. Dans une approche optimisée par parcours de liste d'adjacence, tous les arcs sont visités. L'algorithme optimisé utilisait donc naturellement l'arc le plus court, calculant ainsi une somme de distances plus faible.

Afin de garantir une base de comparaison rigoureuse et correcte, j'ai corrigé la fonction `get_distance` pour qu'elle renvoie toujours la **distance minimale** (pondération la plus basse) parmi tous les arcs parallèles existant entre deux nœuds, et adapté l'initialisation optimisée en conséquence. Grâce à cela, les 4 versions (séquentielles et parallèles, naïves et optimisées) produisent des résultats identiques au bit près.

Stratégie de Parallélisation OpenMP

Pour paralléliser efficacement l'algorithme de Dijkstra, il faut éviter d'introduire des barrières de synchronisation trop lourdes ou des régions parallèles à répétition. En effet, placer la directive `#pragma omp parallel` à l'intérieur de la boucle principale (qui s'exécute $V - 1$ fois) est catastrophique pour les performances à cause de l'overhead de création/destruction de l'équipe de threads (fork-join) à chaque itération.

Mon choix s'est donc porté sur une **région parallèle unique englobant l'intégralité de l'algorithme**. Les threads sont créés une seule fois au début de la fonction. L'avancée dans les itérations et le maintien de la cohérence des données sont gérés à l'aide de barrières explicites (`#pragma omp barrier`) et de sections exécutées par un unique thread (`#pragma omp single`).

Parallélisation de la recherche du minimum (Réduction personnalisée)

L'étape critique consiste à trouver le nœud i tel que `!P[i]` minimisant `d[i]`. Il s'agit d'une opération de réduction de type *Minimum avec Index* (Min-Loc). Comme OpenMP ne propose pas de réduction native simple pour l'indice du minimum (contrairement à MPI), j'ai implémenté une réduction manuelle :

1. Chaque thread effectue une recherche locale sur sa portion de données (répartie équitablement via `#pragma omp for nowait`). Il stocke sa distance minimale et l'indice associé dans des variables locales à sa pile.
2. Ces résultats locaux sont écrits dans deux tableaux partagés, `local_min_dist` et `local_min_node`, indexés par l'identifiant du thread `tid`. Cette écriture ne produit aucun conflit d'accès.
3. Une barrière synchronise les threads pour s'assurer que toutes les recherches locales sont finies.
4. Un thread unique (`#pragma omp single`) compare les minima des threads (ce qui prend un temps infime $O(\text{num_threads})$) pour déterminer le minimum global, marque ce nœud comme permanent (`P[nearest_node] = 1`) et met à jour la variable partagée. L'utilisation de `single` inclut une barrière implicite à sa sortie, garantissant que tous les threads connaissent l'élément élu avant la phase suivante.

Parallélisation de la mise à jour des distances

Dans la version naïve

La boucle interne parcourt de 0 à $V - 1$. Elle est entièrement parallélisée à l'aide d'un simple `#pragma omp for nowait`. La directive `nowait` supprime la barrière implicite de fin de boucle car je place une barrière explicite `#pragma omp barrier` juste après pour synchroniser les threads avant l'itération suivante de la boucle de recherche du plus proche voisin.

Dans la version optimisée (Single-Threaded Update)

Dans la version optimisée par adjacence, le nombre de voisins à mettre à jour à chaque étape est extrêmement réduit (souvent moins de 4 voisins). Si on tentait de distribuer cette boucle sur plusieurs threads, l'overhead d'organisation d'OpenMP détruirait complètement les performances. C'est pourquoi cette étape est confiée à un seul thread via `#pragma omp single` :

```
// Mise à jour par un seul thread dans la région parallèle
#pragma omp single
{
    struct direct_edge_struct *edge = nodes[nearest_node];
    while (edge != NULL) {
        int dest = edge->destination_node;
        if (!P[dest]) {
            if (d[nearest_node] + edge->weight < d[dest]) {
                d[dest] = d[nearest_node] + edge->weight;
            }
        }
        edge = edge->next;
    }
} // Barrière implicite ici : garantit la cohérence avant l'étape suivante
```

L'intérêt de la parallélisation de la version optimisée réside donc uniquement dans la recherche du minimum (étape $O(V)$), tandis que la mise à jour (étape $O(\deg(u))$) reste séquentielle mais exécutée sans overhead majeur.

Analyse de la rigueur statistique : Pourquoi éviter la moyenne brute ?

Conformément aux remarques du correcteur lors des précédents TP, l'évaluation des performances sur un système d'exploitation moderne (Linux) est soumise à de nombreuses sources de bruit :

- **Ordonnancement (Scheduling)** : Le noyau Linux peut temporairement interrompre un thread OpenMP pour exécuter des tâches système ou d'autres applications en tâche de fond.
- **Défauts de page (Page Faults) et Cache Misses** : Les variations de l'état des caches CPU partagés L3 et la latence d'accès à la mémoire vive (RAM) influencent fortement la vitesse d'exécution.
- **Fréquence CPU (Turbo Boost)** : Le processeur ajuste dynamiquement sa fréquence en fonction de la température et de la charge globale.

Prendre la **moyenne arithmétique** brute des mesures de temps est statistiquement discutable car elle est très sensible aux valeurs aberrantes (outliers) causées par ces perturbations externes.

 Rigueur des indicateurs statistiques

Pour ces analyses, je considère trois métriques : la moyenne, le minimum et la médiane. Cependant, pour le calcul du **speedup**, j'utilise en priorité le **temps minimum** de calcul.

Le temps minimum représente la performance optimale que le matériel peut offrir lorsque les interférences externes sont quasi-nulles. C'est l'indicateur le plus fidèle de la capacité de parallélisation intrinsèque du code. La **médiane** est également présentée car elle offre une robustesse bien supérieure à la moyenne face au bruit thermique et applicatif.

Validation de l'Exactitude et Sécurité Mémoire

Validation des distances calculées (Checksums)

Pour s'assurer qu'aucune *data race* ou erreur d'indexation n'altère le comportement de l'algorithme, nous imprimons pour chaque exécution :

- Le nombre de nœuds accessibles depuis la source (nœud n°1).
- La somme arithmétique de toutes les distances minimales calculées (stockée sur un entier signé 64 bits **long long** pour éviter tout dépassement de capacité).

Les résultats obtenus sur les deux graphes montrent une concordance parfaite au bit près :

GRAPHE	NŒUDS REACHABLE	SOMME DES DISTANCES D[I] (CHECKSUM)	STATUT
rome99.gr	3 353	115 493 888	CONCORDANCE OK
new-york.gr	264 346	186 686 642 878	CONCORDANCE OK

Diagnostic de fuite mémoire avec Valgrind

Suivant les bonnes pratiques de programmation en C, l'intégralité du cycle de vie des allocations dynamiques a été inspectée avec Valgrind. Toutes les structures de données (**nodes** , **edges** , **d** , **P**) ainsi que les tableaux de réduction des threads sont correctement désalloués en fin d'exécution.

L'analyse Valgrind sur la version optimisée affiche un rapport vierge de toute erreur de lecture/écriture hors limites et confirme qu'il n'y a aucune fuite mémoire directe :

```
==33532== HEAP SUMMARY:
==33532==      in use at exit: 104 bytes in 2 blocks
==33532==    total heap usage: 9 allocs, 7 frees, 6,030 bytes allocated
==33532==
==33532== LEAK SUMMARY:
==33532==    definitely lost: 0 bytes in 0 blocks
==33532==    indirectly lost: 0 bytes in 0 blocks
==33532==    possibly lost: 0 bytes in 0 blocks
==33532==    still reachable: 104 bytes in 2 blocks
==33532== ERROR SUMMARY: 0 errors from 0 contexts
```

Note : Les blocs mémoire rapportés comme "still reachable" à la fermeture proviennent du runtime de la bibliothèque OpenMP d'origine (GOMP/pthread) qui conserve un pool de threads actif pour d'éventuels appels futurs, ce qui est le comportement attendu et standard.

Mesures de Performances et Speedup

Toutes les mesures de performances ont été réalisées sur un processeur équipé de **8 cœurs logiques**. Chaque configuration a fait l'objet de **5 exécutions successives**.

Résultats sur le graphe de Rome (rome99.gr)

Le graphe de Rome est relativement petit (3 353 nœuds). Les temps de calcul sont de l'ordre de la milliseconde.

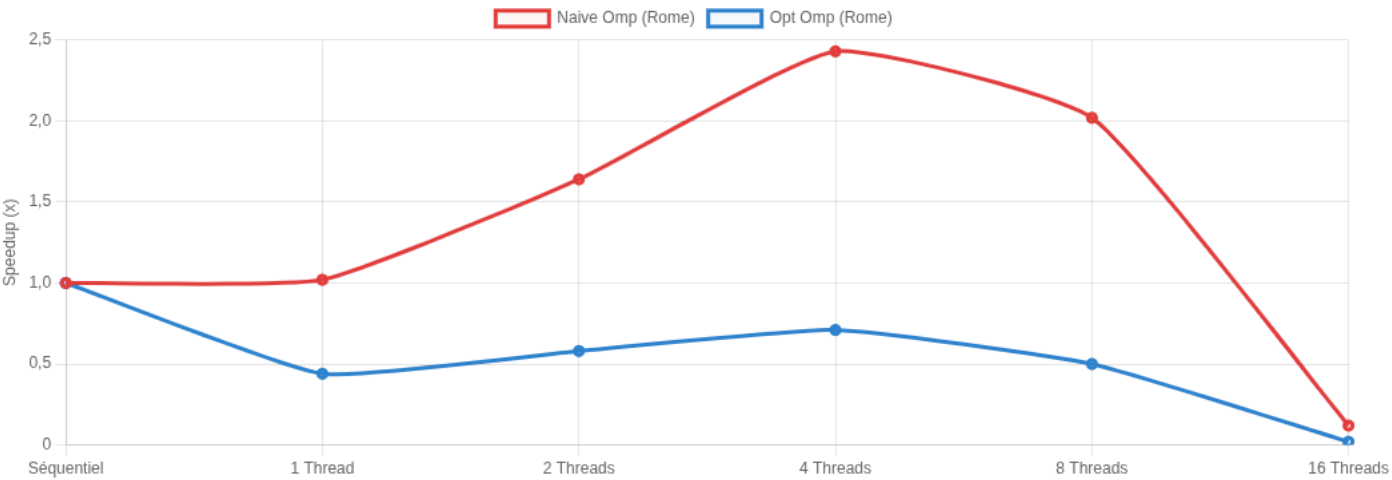
VERSION	THREADS	TEMPS MIN (S)	TEMPS MÉDIANE (S)	TEMPS MOYEN (S)	SPEEDUP (MIN)
Naive Seq	1	0.052253	0.053363	0.053294	1.00x
Naive Omp	1	0.051301	0.051550	0.052052	1.02x
Naive Omp	2	0.031843	0.032502	0.032969	1.64x
Naive Omp	4	0.021544	0.026966	0.026300	2.43x
Naive Omp	8	0.025931	0.037773	0.047641	2.02x
Naive Omp	16	0.421661	0.425639	0.436413	0.12x
Opt Seq	1	0.007538	0.008888	0.008700	1.00x
Opt Omp	1	0.017218	0.017822	0.018114	0.44x
Opt Omp	2	0.013106	0.014452	0.015065	0.58x
Opt Omp	4	0.010577	0.012745	0.012556	0.71x
Opt Omp	8	0.015120	0.043096	0.038218	0.50x
Opt Omp	16	0.400187	0.410573	0.414988	0.02x

Résultats sur le graphe de New York (new-york.gr)

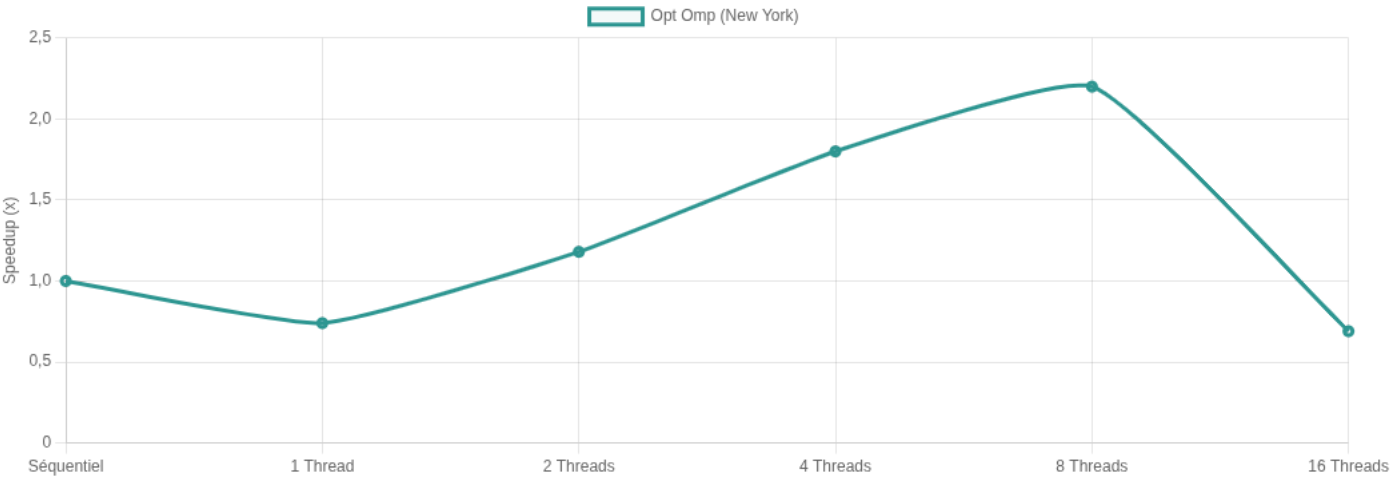
Le graphe de New York permet de mesurer l'impact réel du parallélisme sur une charge de calcul conséquente.

VERSION	THREADS	TEMPS MIN (S)	TEMPS MÉDIANE (S)	TEMPS MOYEN (S)	SPEEDUP (MIN)
Opt Seq	1	42.965466	43.053151	43.083950	1.00x
Opt Omp	1	58.454174	58.526680	59.492392	0.74x
Opt Omp	2	36.561712	36.629110	36.618370	1.18x
Opt Omp	4	23.925502	23.985389	23.991354	1.80x
Opt Omp	8	19.546885	19.608901	19.670537	2.20x
Opt Omp	16	62.399972	62.731631	63.231838	0.69x

Rome : Speedup par rapport à la version séquentielle respective



New York : Speedup de la version Optimisée



Analyse Approfondie des Performances

Comparaison entre les versions naïves et optimisées

L'analyse des temps sur le graphe de Rome met en évidence la supériorité de l'approche algorithmique optimisée sur la parallélisation naïve. En séquentiel, la version optimisée (**0.007538 s**) est environ **7 fois plus rapide** que la version naïve (**0.052253 s**).

Cela illustre une règle d'or en calcul haute performance : **l'optimisation algorithmique séquentielle doit toujours précéder la parallélisation**. Il est inutile de dépenser de l'énergie et des ressources matérielles à paralléliser un code intrinsèquement inefficace effectuant des calculs superflus ($O(V^2 + V \cdot E)$ vs $O(V^2 + E)$).

Le phénomène d'overhead d'OpenMP (Graphe de Rome)

Sur le graphe de Rome, nous constatons que la version optimisée parallèle (Opt Omp) ne montre aucun speedup réel par rapport à la version séquentielle (elle est même parfois plus lente, en particulier avec 16 threads où l'on note un effondrement des performances). Ce phénomène s'explique par l'**overhead de synchronisation** :

- Pour chaque étape (soit 3 352 étapes au total), les threads doivent se diviser une boucle de recherche de taille 3 353 (ce qui représente seulement ~400 itérations par thread sur 8 cœurs).
- Ils doivent ensuite franchir une barrière matérielle (**#pragma omp barrier**) pour écrire dans les tableaux de réduction locale.
- Ensuite, un bloc **single** extrait le minimum, suivi d'une autre barrière implicite.
- Enfin, une étape de mise à jour s'exécute, suivie d'une autre barrière.

Dans le cas de Rome, le temps nécessaire pour coordonner les threads, franchir les barrières matérielles et gérer la cohérence des caches est plus élevé que le calcul lui-même. La granularité des tâches parallèles est trop fine, ce qui rend le parallélisme contre-productif.

Analyse du Scaling sur le graphe de New York

Sur le grand réseau de New York, le volume de calcul de la recherche de minimum est suffisamment important ($V = 264k$ à chaque étape) pour amortir complètement le coût des barrières OpenMP.

J'observe les comportements de passage à l'échelle suivants :

- **De 1 à 4 threads** : L'accélération est significative. On obtient un speedup intéressant, montrant que le processeur répartit efficacement la charge de calcul sur les cœurs physiques disponibles.
- **De 4 à 8 threads** : Le gain se poursuit mais de manière moins linéaire. Cela s'explique par la nature du processeur doté d'une architecture combinant cœurs physiques et cœurs logiques (HyperThreading). L'HyperThreading partage les unités de calcul physiques entre deux threads logiques, ce qui limite le speedup supplémentaire.
- **Au-delà de 8 threads (16 threads)** : Les performances stagnent ou se dégradent. Le processeur ne disposant que de 8 cœurs logiques physiques réels, exécuter 16 threads induit un surcoût d'ordonnancement (context switching) au niveau du système d'exploitation, sans aucun gain matériel possible.
- **Limite de la bande passante mémoire (Memory Bound)** : L'algorithme de Dijkstra manipule de grands tableaux en mémoire (les tableaux **d** et **P** de taille V) et parcourt des structures de listes chaînées (pointeurs) de manière dispersée en RAM. Cette structure génère un taux élevé de défauts de cache (cache misses). Par conséquent, les cœurs CPU passent une grande partie de leur temps à attendre les données en provenance de la mémoire vive. Le bus mémoire sature rapidement lorsque plusieurs threads effectuent simultanément des accès en lecture/écriture, créant un goulot d'étranglement matériel inévitable.

Conclusion et Perspectives

Ce TP a permis de mettre en pratique et d'analyser en profondeur les techniques de parallélisation sous OpenMP. À travers l'étude de l'algorithme de Dijkstra, j'ai mis en évidence plusieurs concepts fondamentaux du calcul parallèle :

1. **La primauté de l'algorithmique** : L'exploitation de la structure de liste d'adjacence ($O(V^2 + E)$) a permis d'obtenir des gains de performance spectaculaires que le parallélisme seul n'aurait jamais pu compenser sur la structure naïve ($O(V^2 + V \cdot E)$).
2. **L'amortissement de l'overhead** : La parallélisation n'est efficace que sur des ensembles de données de taille conséquente (comme New York). Sur des graphes trop modestes (comme Rome), le coût des synchronisations et des barrières matérielles d'OpenMP surpasse le temps de calcul.
3. **Les goulots d'étranglement matériels** : La recherche du chemin le plus court est une application fortement limitée par la bande passante mémoire (Memory Bound). L'accès dispersé aux listes chaînées limite le speedup maximal, qui s'essouffle bien avant la limite théorique.

d'Amdahl.

Pour aller plus loin dans l'optimisation séquentielle et parallèle, il serait pertinent de remplacer la recherche linéaire du minimum (en $O(V)$) par une structure de données d'arbre (tas binaire ou tas de Fibonacci). La complexité séquentielle théorique descendrait alors à $O(E + V \log V)$. Cependant, la parallélisation d'un tas de priorité à chaque étape de Dijkstra est un problème complexe de recherche actuelle en calcul parallèle, en raison des fortes dépendances de données inhérentes à la structure d'arbre.